



UNIVERSITY OF PISA
MASTER'S DEGREE IN COMPUTER SCIENCE,
ARTIFICIAL INTELLIGENCE

**Computational Mathematics for Learning &
Data Analysis (646AA)**

Members:
Matthew Bernardi
Gabriele Marsili

ACADEMIC YEAR: 2025/2026

Contents

1	Introduction to the problem	2
1.1	Extreme Learning Machine	2
1.2	Least-Squares Problem	2
1.3	L_1 regularization	4
2	Algorithm 1: Iteratively Reweighted Least Squares	5
3	Algorithm 2: Deflected Subgradient Method	6
3.1	Motivation: limitations of the plain subgradient method	6
3.1.1	The deflection idea	7
3.2	Convergence conditions: balancing deflection and stepsize	7
3.2.1	Optimal deflection parameter	8
3.3	The target-level mechanism	9
3.3.1	The complete algorithm	9
3.3.2	Parameters	9
3.4	Application to our problem	10
3.4.1	Subgradient computation	10
3.4.2	Convergence	11
3.4.3	Expected properties	13
4	Comparison between the two Algorithms	15
5	Implementation	16
6	Results	17
7	Conclusions	18

Chapter 1

Introduction to the problem

The goal of this project is to analyze the problem of learning of an **Extreme Learning Machine** (ELM) algorithm, solving the **least-squares problem** with L_1 **regularization** using the **Iteratively Reweighted Least Squares** and **Deflected Subgradient Method** algorithms.

In this section, we will cover the background needed to solve this problem, and we will leave the learning algorithms' explanation in their respective chapters.

1.1 Extreme Learning Machine

Extreme Learning Machine (ELM) is a machine learning model implemented as a neural network with a single hidden layer, in which:

- $\mathbf{W}_1 \in \mathbb{R}^{H \times N}$ is the input-to-hidden weight matrix (where H is the number of nodes in the hidden layer and N is the number of input nodes) that is **randomly generated** and **static** (does not change during training).
- $\sigma(\cdot)$ is the element-wise activation function applied to the $\mathbf{W}_1 \mathbf{x} \in \mathbb{R}^H$ vector, where $\mathbf{x} \in \mathbb{R}^N$ is the input vector.
- $\mathbf{w} \in \mathbb{R}^H$ is the hidden-to-output weight vector, chosen by solving the least-squares problem.
- $\hat{y} = \mathbf{w}^T \sigma(\mathbf{W}_1 \mathbf{x}) \in \mathbb{R}$ is the prediction of the model.

1.2 Least-Squares Problem

The Least-Squares Problem is an optimization problem used in machine learning for data fitting. It helps finding the best curve that minimizes the sum of squared differences between the *predicted value* $A\mathbf{x}$ and the observed *ground truth* (or *target*) $\mathbf{y}$. Mathematically, we can represent this problem as follows:

$$\min_{\mathbf{x}} ||A\mathbf{x} - \mathbf{y}||$$

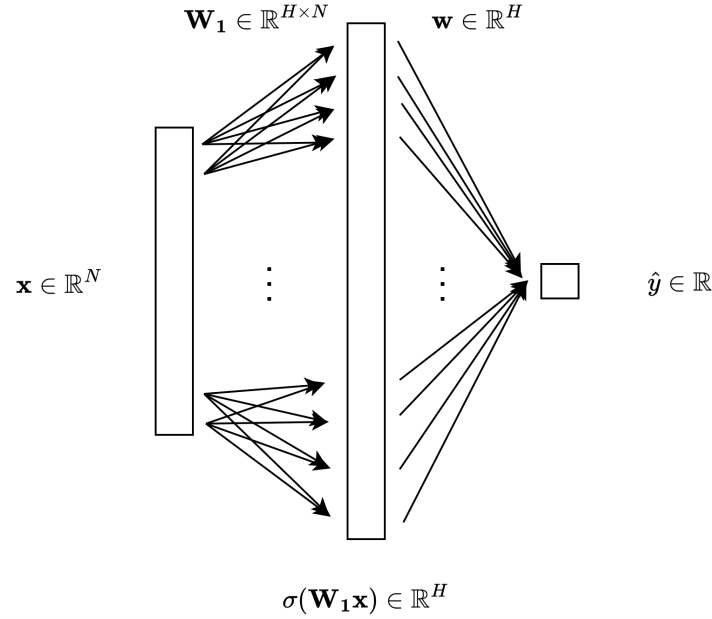


Figure 1.1: Visual representation of an Extreme Learning Machine model.

In our case, a more suited definition is the following one, considering $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1) \in \mathbb{R}^{M \times H}$ the matrix of activations of the hidden layer, while $\mathbf{X}_{\text{origin}} \in \mathbb{R}^{M \times N}$ is the matrix of all the inputs, and $\mathbf{y} \in \mathbb{R}^M$ is the vector of all the target values:

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

The Least-Squares problem is proven to have a unique solution if and only if the matrix A has full column rank. In that case, we can write the optimization problem as follows:

$$\min_{\mathbf{w}} \frac{1}{2} \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{y}^T \mathbf{X} \mathbf{w} + \frac{1}{2} \mathbf{y}^T \mathbf{y}$$

since the gradient of this function is $\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y}$, the Hessian matrix is $\mathbf{X}^T \mathbf{X} \succ 0$, which means that the function is strictly convex. In this case, the minimum exists and is unique, and can be found solving the equation:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y} = 0$$

or

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

where $\mathbf{X}^T \mathbf{X}$ is square invertible since it's positive definite. The linear system has a unique solution. We can solve this equation using methods such as *IRLS* and *Deflected Subgradient Methods*.

1.3 L_1 regularization

When solving the Least-Squares Problem, we may prefer solutions that use a subset of available features. This prevents the problem of overfitting the model to the data, and producing a result that is more robust with respect to the data given. Regularization is the technique used to add a penalty to weights.

The L_1 regularization, also called Lasso, tends to bring many weights to zero. This is a form of implicit *feature selection* mechanism. The L_1 regularization is the best convex approximation for the L_0 norm, which penalizes the number of non-zero weights.

When adding the L_1 regularization term to the Least-Squares problem, the equation is the following:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1$$

where $\lambda > 0 \in \mathbb{R}$ is an hyperparameter to balance sparsity (simplicity) and precision: the higher the λ , the more sparse the $\mathbf{w}$ solution vector.

The price to pay when using the Lasso regularization is the non-differentiability of the loss function $f(\mathbf{w})$ for each point where $w_i = 0$ for at least one i index. This is because of the gradient, which can be computed correctly only in the points where $w_i \neq 0 \ \forall i$, and is the following:

$$\nabla f(\mathbf{w}) = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \text{sign}(\mathbf{w})$$

Because of that, we can use the Deflected Subgradient Methods (1)

Chapter 2

Algorithm 1: Iteratively Reweighted Least Squares

Chapter 3

Algorithm 2: Deflected Subgradient Method

Notation. In this chapter we present the deflected subgradient method in its general form, using x^i for the iterates. When applied to our specific LASSO problem (Section 3.4.1 onward), the iterates are denoted w_i to match the notation of Chapter 1.

We choose the *stepsize-restricted* variant with *target level* (SGPTL) and *optimal deflection*, because it avoids the need for a priori knowledge of f^* while retaining the practical efficiency of the Polyak stepsize, and the optimal deflection minimizes the direction norm, maximising progress per iteration.

3.1 Motivation: limitations of the plain subgradient method

The starting point for this algorithm is the *subgradient method*, the natural extension of the gradient method to convex nondifferentiable functions. Given $g^i \in \partial f(x^i)$, the update rule mirrors that of the gradient method:

$$x^{i+1} = x^i - \alpha^i g^i. \quad (3.1)$$

However, this method suffers from two structural limitations. First, it uses only local information (the subgradient at the current point) and retains no memory of previous iterations. The result is a pronounced zig-zag behavior, where the algorithm oscillates between opposing directions, leading to very slow convergence. Second, the stepsize α^i must be chosen with great care. A *constant* stepsize does not work: the algorithm cycles forever without converging. The stepsize must tend to zero, but not too fast: the standard requirement is the *Diminishing-Square Summable* (DSS) condition

$$\sum_{i=1}^{\infty} \alpha^i = \infty \quad \text{and} \quad \sum_{i=1}^{\infty} (\alpha^i)^2 < \infty, \quad (3.2)$$

satisfied, for example, by $\alpha^i = c/i$. The DSS rule guarantees convergence, but the practical speed is very poor. These issues are analogous to the well-known zig-zag of the steepest-descent method on ill-conditioned smooth functions, which is cured by *conjugate directions* (Heavy Ball, conjugate gradient). In the nonsmooth setting, the corresponding remedy is **deflection**.

Intrinsic difficulty of nonsmooth optimization.

Before introducing the algorithm, it is important to understand the fundamental complexity gap between smooth and nonsmooth convex optimization:

Regularity	Convexity	Complexity
$f \in C^1$, L -smooth	τ -strongly convex	$O(\log(1/\varepsilon))$
$f \notin C^1$, L -Lipschitz	τ -strongly convex	$\Omega(L^2/\varepsilon)$
$f \in C^1$, L -smooth	convex	$O(1/\sqrt{\varepsilon})$
$f \notin C^1$, L -Lipschitz	convex	$\Omega(L/\varepsilon^2)$

The gap between the smooth strongly convex case ($O(\log(1/\varepsilon))$) and the nonsmooth convex case ($\Omega(1/\varepsilon^2)$) is enormous: to reach $\varepsilon = 10^{-6}$, the former requires ≈ 14 iterations while the latter requires $\approx 10^{12}$.

3.1.1 The deflection idea

Deflected subgradient methods replace the raw subgradient g^i with a *deflected direction* d^i , defined as a convex combination of the current subgradient and the previous direction:

$$d^i = \gamma^i g^i + (1 - \gamma^i) d^{i-1}, \quad d^0 = g^0, \quad (3.3)$$

where $\gamma^i \in (0, 1]$ is the **deflection parameter**. The iterate is then updated as

$$x^{i+1} = x^i - \alpha^i d^i. \quad (3.4)$$

This can be seen as a nonsmooth analogue of the conjugate gradient method, where the deflected direction d^i plays the role of the conjugate direction. The parameter γ^i controls the trade-off between responsiveness and memory:

- $\gamma^i = 1$: only the current subgradient is used (plain subgradient method). Maximum local responsiveness, but oscillatory behavior.
- γ^i small: the direction is dominated by d^{i-1} , producing a smoother trajectory at the risk of ignoring new information.
- Intermediate γ^i : the best compromise between reactivity and smoothness.

3.2 Convergence conditions: balancing deflection and stepsize

The joint choice of γ^i (deflection) and α^i (stepsize) must be carefully coordinated: if both are too large the algorithm may diverge. Two complementary frameworks guarantee convergence.

Approach 1: stepsize-restricted. The deflection γ^i is chosen freely, and the stepsize is constrained via a Polyak-type rule:

$$\alpha^i = \frac{\beta^i (f(x^i) - f^*)}{\|d^i\|^2}, \quad \beta^i \leq \gamma^i. \quad (3.5)$$

Since f^* is unknown in practice, it is replaced by the target level $f_{\text{ref}} - \delta$ (see Section 3.3). The condition $\beta^i \leq \gamma^i$ ensures that the step shrinks as the deflection grows stronger (γ^i small). The rationale is that a large deflection injects the “old” direction d^{i-1} into d^i , which may no longer be a descent direction; a shorter step compensates for this risk.

Approach 2: deflection-restricted. The stepsize α^i is chosen a priori (e.g. via a DSS rule), and the deflection is constrained from below:

$$\gamma^i \geq \frac{\alpha^{i-1} \|d^{i-1}\|^2}{(f(x^i) - f^*) + \alpha^{i-1} \|d^{i-1}\|^2}. \quad (3.6)$$

Again, f^* is replaced by the target level in the actual implementation. As the algorithm converges ($f(x^i) \rightarrow f^*$), the term $(f(x^i) - f^*)$ in the denominator vanishes, so the right-hand side tends to 1, i.e. $\gamma^i \rightarrow 1$: near the solution the deflection disappears and the method reverts to the plain subgradient. This is consistent: deflection accelerates the trajectory far from the optimum, but close to it the local subgradient information is preferable.

3.2.1 Optimal deflection parameter

At each iteration, the deflection parameter can be chosen to minimize the norm of the deflected direction:

$$\gamma^i \in \arg \min_{\gamma \in [0,1]} \|\gamma g^i + (1 - \gamma) d^{i-1}\|^2. \quad (3.7)$$

In the Polyak stepsize (3.5) the denominator is $\|d^i\|^2$; minimising it maximises the step length and hence the progress per iteration. We therefore seek the “most efficient” direction among all convex combinations of g^i and d^{i-1} . Problem (3.7) is a quadratic in a single variable constrained to $[0, 1]$ and admits a closed-form solution. Define $\varphi(\gamma) = \|\gamma g + (1 - \gamma) d^-\|^2$:

$$\varphi(\gamma) = \gamma^2 \|g\|^2 + 2\gamma(1 - \gamma) \langle g, d^- \rangle + (1 - \gamma)^2 \|d^-\|^2. \quad (3.8)$$

Setting $\varphi'(\gamma) = 0$ gives the unconstrained minimizer

$$\gamma^* = \frac{\|d^-\|^2 - \langle g, d^- \rangle}{\|g - d^-\|^2}, \quad (3.9)$$

and the constrained solution is $\gamma^i = \min(1, \max(0, \gamma^*))$.

3.3 The target-level mechanism

The Polyak stepsize in its original form requires the optimal value f^* , which is precisely the quantity we are trying to compute. Recall the **fundamental relationship** for the subgradient method:

$$\|x^{i+1} - x^*\|^2 \leq \|x^i - x^*\|^2 \underbrace{- 2\alpha^i(f(x^i) - f^*)}_{\leq 0 \text{ (good)}} + \underbrace{(\alpha^i)^2 \|g^i\|^2}_{\geq 0 \text{ (bad)}}. \quad (3.10)$$

The choice $\alpha^i = (f(x^i) - f^*)/\|g^i\|^2$ is the one that maximises the guaranteed reduction in distance to x^* at each step; however, it requires knowledge of f^* . The **target-level** mechanism circumvents this by maintaining an estimate $f_{\text{lev}} = f_{\text{ref}} - \delta$ of f^* , where:

- f_{ref} is the best function value found so far (the *record value*);
- $\delta > 0$ is an expected improvement margin—the “hope” of how much further we can improve.

If the estimate is accurate ($f_{\text{ref}} - \delta \approx f^*$), the algorithm makes good progress. If the estimate is too optimistic ($f_{\text{ref}} - \delta < f^*$), the Polyak numerator $f(x^i) - f_{\text{lev}}$ becomes very small or negative, producing tiny or ill-defined steps and stalling the algorithm. When this is detected—after a prescribed amount of travel without significant improvement— δ is reduced (multiplied by $\rho < 1$) and the algorithm retries with a less ambitious target.

3.3.1 The complete algorithm

The full Deflected Subgradient Method with target level is given in Algorithm 1.

The algorithm maintains three key quantities (we write $\tilde{f}$ in the pseudocode; in the analysis below, $\tilde{f}^i$ denotes the record value at iteration i):

- $\tilde{f}^i = \min_{h \leq i} f(w_h)$: the **record value**—the best f encountered up to iteration i . Since the method is non-monotone, this is the quantity that converges and constitutes the effective output.
- $f_{\text{ref}} - \delta$: the **target level**—the current estimate of f^* used in the Polyak stepsize.
- r : a counter of distance travelled without significant improvement. When r exceeds the patience threshold R , the target is deemed too optimistic and δ is reduced.

3.3.2 Parameters

Remark 3.1 (Parameter roles). The algorithm involves several parameters whose calibration is critical for performance:

- δ_0 (*initial gap estimate*): an estimate of $f(w_0) - f^*$. If set too large, the target $f_{\text{ref}} - \delta$ is unrealistically low, producing oversized steps and instability; if too small, the steps are insufficient to make meaningful progress. A reasonable choice is $\delta_0 = c \cdot f(w_0)$ with $c \in [0.01, 0.5]$.

Algorithm 1 Deflected Subgradient with Target Level (SGPTL)

Require: f , w_0 , $i_{\max}$, $\beta \in (0, 2)$, $\delta_0 > 0$, $R > 0$, $\rho \in (0, 1)$

```
1:  $r \leftarrow 0$ ;  $\delta \leftarrow \delta_0$ ;  $f_{\text{ref}} \leftarrow \bar{f} \leftarrow f(w_0)$ ;  $d_{-1} \leftarrow 0$ 
2: for  $i = 0, 1, \dots, i_{\max}$  do
3:   compute  $g \in \partial f(w_i)$  (any subgradient)
4:   compute  $\gamma_i$  via (3.7) (optimal deflection)
5:    $d_i \leftarrow \gamma_i g + (1 - \gamma_i) d_{i-1}$  (deflected direction)
6:    $\alpha_i \leftarrow \beta (f(w_i) - (f_{\text{ref}} - \delta)) / \|d_i\|^2$  (Polyak with target level)
7:    $w_{i+1} \leftarrow w_i - \alpha_i d_i$  (step)
8:   if  $f(w_{i+1}) \leq f_{\text{ref}} - \delta/2$  then (significant improvement)
9:      $f_{\text{ref}} \leftarrow \bar{f}$ ;  $r \leftarrow 0$ 
10:  else if  $r > R$  then (stalled: reduce target)
11:     $\delta \leftarrow \delta \rho$ ;  $r \leftarrow 0$ 
12:  else
13:     $r \leftarrow r + \alpha_i \|d_i\|$  (accumulate travelled distance)
14:  end if
15:   $\bar{f} \leftarrow \min(\bar{f}, f(w_{i+1}))$  (update record)
16: end for
17: return  $w_{\text{best}}$  (the iterate achieving  $\bar{f}$ )
```

- $\rho \in (0, 1)$ (*target reduction rate*): controls how quickly δ is reduced when progress stalls. Typical values are $\rho = 0.95$ (slow, more iterations but tighter target tracking) or $\rho = 0.9$ (more aggressive reduction).
- $R > 0$ (*patience threshold*): the distance travelled without improvement before δ is reduced. Too small causes premature reductions (overly conservative target); too large delays adaptation.
- $\beta \in (0, 2)$ (*step modulation*): scales the Polyak step. The natural choice is $\beta = 1$. Values in $(1, 2)$ yield more aggressive (but less stable) steps; values in $(0, 1)$ are more conservative.

3.4 Application to our problem

3.4.1 Subgradient computation

To apply the SGPTL algorithm to our problem

$$f(w) = \|Xw - y\|_2^2 + \lambda \|w\|_1, \quad (3.11)$$

we need to compute a subgradient $\tilde{g} \in \partial f(w)$ at each iteration. Recall from the introduction that $f = q + h$, where $q(w) = \|Xw - y\|_2^2$ is the smooth part and $h(w) = \lambda \|w\|_1$ is the nonsmooth part. Since both are convex, the sum rule for subdifferentials gives

$$\partial f(w) = \nabla q(w) + \lambda \partial \|w\|_1,$$

where $\nabla q(w) = 2X^\top(Xw - y)$ is the gradient of the smooth part (which exists everywhere), and $\partial \|w\|_1$ is the subdifferential of the ℓ_1 norm, computed componentwise. A subgradient of f at w is therefore

$$\tilde{g} = 2X^\top(Xw - y) + \lambda s, \quad s_i \in \partial |w_i|, \quad (3.12)$$

where

$$s_i = \begin{cases} +1 & \text{if } w_i > 0, \\ -1 & \text{if } w_i < 0, \\ \in [-1, +1] & \text{if } w_i = 0. \end{cases}$$

When $w_i = 0$ the standard choice is $s_i = 0$, which corresponds to the minimum-norm element of $\partial |w_i|$ and yields the steepest-descent direction ($s_* = -\arg \min\{\|s\| : s \in \partial f(x)\}$). In Algorithm 1 and throughout this chapter, g^i denotes the subgradient $\tilde{g}$ evaluated at the i -th iterate w_i .

Computational cost. The subgradient computation proceeds as follows:

1. Compute the residual $r = Xw - y$ $O(mn)$
2. Compute $\nabla q = 2X^\top r$ $O(mn)$
3. Compute s componentwise: $s_i = \text{sign}(w_i)$ if $w_i \neq 0$, $s_i = 0$ otherwise $O(n)$
4. Assemble $\tilde{g} = \nabla q + \lambda s$ $O(n)$

The total cost per iteration is $O(mn)$, dominated by the two matrix-vector products. This is significantly cheaper than the $O(n^3/3)$ per-iteration cost of IRLS (Cholesky factorization), or $O(kn^2)$ when using k steps of conjugate gradient as the inner solver.

3.4.2 Convergence

Theorem 3.2 (Convergence of the Deflected Subgradient Method).

Let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be convex and let $\{w_i\}$ be the sequence generated by Algorithm 1 with the convergence conditions satisfied (stepsize-restricted with $\beta^i \leq \gamma^i$, or deflection-restricted). Assume that the subgradients are uniformly bounded, i.e. there exists $L > 0$ such that $\|g^i\| \leq L$ for all i . Then the sequence of record values

$$\bar{f}^i = \min_{h \leq i} f(w_h)$$

converges to the optimal value: $\bar{f}^i \rightarrow f^$ as $i \rightarrow \infty$. The worst-case complexity is $O(1/\varepsilon^2)$: to achieve $\bar{f}^i - f^* \leq \varepsilon$, at most $O(L^2/\varepsilon^2)$ iterations are required.*

Proof sketch.

The argument builds on the **fundamental relationship** (3.10), adapted to the deflected case. For the plain subgradient $x^{i+1} = x^i - \alpha^i g^i$, expanding the squared distance gives

$$\begin{aligned} \|x^{i+1} - x^*\|^2 &= \|x^i - x^*\|^2 - 2\alpha^i \langle g^i, x^i - x^* \rangle + (\alpha^i)^2 \|g^i\|^2 \\ &\leq \|x^i - x^*\|^2 - 2\alpha^i (f(x^i) - f^*) + (\alpha^i)^2 \|g^i\|^2, \end{aligned} \quad (3.13)$$

where the inequality uses the subgradient inequality $f(x^*) \geq f(x^i) + \langle g^i, x^* - x^i \rangle$, i.e. $\langle g^i, x^i - x^* \rangle \geq f(x^i) - f^*$. In the deflected case the direction d^i replaces g^i in (3.13). The convergence conditions (stepsize-restricted or deflection-restricted) ensure that an analogous relationship holds for the deflected direction, namely that the “good” term $-2\alpha^i(f(x^i) - f^*)$ dominates the “bad” term $(\alpha^i)^2 \|d^i\|^2$ on average. Under the Polyak stepsize with the condition $\beta^i \leq \gamma^i$:

1. The record value $\bar{f}^i$ is non-increasing by construction.
2. Telescoping (3.13) over $i = 1, \dots, k$ and using $\|x^{k+1} - x^*\|^2 \geq 0$ yields

$$\bar{f}^k - f^* \leq \frac{L \|x^1 - x^*\|}{\sqrt{k}}, \quad (3.14)$$

which, solving for k , gives $k \geq L^2 \|x^1 - x^*\|^2 / \varepsilon^2 = O(1/\varepsilon^2)$. $\square$

Remark 3.3 (Key observations on convergence).

- **Non-monotonicity.** It is the sequence of record values $\bar{f}^i$ that converges, *not* the sequence of iterates w_i or function values $f(w_i)$. The method is intrinsically non-monotone: $f(w_{i+1}) > f(w_i)$ can and does happen frequently. The fundamental relationship guarantees that the *distance* $\|x^i - x^*\|$ decreases, but the function value may oscillate. For this reason the algorithm returns the best iterate found, not the last one.
- **Optimality of the rate.** The $O(1/\varepsilon^2)$ complexity is identical to that of the plain subgradient method. This is an *intrinsic* limitation of nonsmooth optimization: for convex, L -Lipschitz functions that are not C^1 , the lower bound is $\Omega(L^2/\varepsilon^2)$, so the method is worst-case optimal. Deflection does not improve the worst-case complexity, but significantly improves practical behavior by reducing oscillations.
- **Comparison with smooth methods.** For smooth, strongly convex functions the complexity is $O(\log(1/\varepsilon))$ —exponentially better. Concretely: going from $\varepsilon = 10^{-3}$ to $\varepsilon = 10^{-4}$ with a subgradient method requires $(10^{-3}/10^{-4})^2 = 100 \times$ more iterations.

Verification of hypotheses for our problem. We check that the assumptions of Theorem 3.2 are satisfied by (3.11):

1. **Convexity.** $f(w) = \|Xw - y\|_2^2 + \lambda \|w\|_1$ is convex, being the sum of a convex smooth function (q , with Hessian $2X^\top X \succeq 0$) and a convex norm. If X has full column rank, q is τ -strongly convex with $\tau = 2\lambda_{\min}(X^\top X) > 0$, so f is also strongly convex and the minimum is unique. In this case the theoretical lower bound for nonsmooth optimization improves from $\Omega(1/\varepsilon^2)$ to $\Omega(1/\varepsilon)$ (see the complexity table in Section 3.1). However, the DSM as implemented does not exploit strong convexity, so its worst-case rate remains $O(1/\varepsilon^2)$.

2. **Bounded subgradients.** The ℓ_1 component λs has norm at most $\lambda\sqrt{n}$ (since $|s_i| \leq 1$). The smooth component $2X^\top(Xw - y)$ grows linearly with $\|w\|$, so f is *not* globally Lipschitz on $\mathbb{R}^n$. However, the convergence analysis only requires that the subgradients be uniformly bounded *along the generated sequence*, i.e. $\|g^i\| \leq L$ for all i . In practice this is satisfied: the target-level mechanism prevents arbitrarily large steps, so the iterates remain confined to a bounded set and $\|g^i\|$ is uniformly bounded.

3.4.3 Expected properties

Based on the theoretical analysis, we expect the following behavior:

- **Non-monotone convergence.** The function value $f(w_i)$ oscillates throughout the run. The quantity to monitor is the record value $\bar{f}^i$, which decreases monotonically by construction. In convergence plots it is essential to display $\bar{f}^i$ rather than $f(w_i)$.
- **Sublinear convergence.** In a semilogarithmic plot ($\log(\bar{f}^i - f^*)$ vs. iterations), the curve flattens progressively: initial improvements are rapid, but progress slows at each decade of precision. This contrasts with the typically linear convergence of IRLS, which produces a straight line on the same scale. The reason is the $O(1/\varepsilon^2)$ complexity: doubling the desired precision requires roughly quadrupling the number of iterations.
- **Difficulty at high accuracy.** Achieving gaps $\bar{f}^i - f^* < 10^{-4}$ requires a prohibitive number of iterations, due to the $O(1/\varepsilon^2)$ rate. Concretely: if N iterations suffice for $\varepsilon = 10^{-3}$, roughly $100N$ are needed for $\varepsilon = 10^{-4}$. This is a *theoretical* limitation of the approach, not an implementation defect.
- **Low per-iteration cost.** Each iteration costs $O(mn)$, dominated by the two matrix-vector products. This makes the method competitive in CPU time when low accuracy suffices, particularly for large-scale problems where the $O(n^3)$ cost of IRLS becomes prohibitive.
- **Parameter sensitivity.** Performance depends critically on the calibration of δ_0 , ρ , R , and β . Poor choices may lead to extremely slow convergence (δ too small, insufficient steps) or instability (δ too large, oversized steps). A preliminary tuning phase is recommended.
- **No reliable stopping criterion.** Unlike smooth methods where $\partial f(x^*) = \{\nabla f(x^*)\} = \{0\}$ is a singleton and the gradient norm provides a natural stopping criterion, in the nonsmooth case $\partial f(x^*)$ is an interval containing 0 but also elements with large norm, so $\|g^i\|$ can remain large even at the optimum (recall from Chapter 1 that $0 \in \partial f(w^*)$ is the optimality condition, but other elements of $\partial f(w^*)$ need not be small). The only practical criterion is to monitor the record value and stop after a prescribed iteration budget or when $\bar{f}^i$ ceases to improve appreciably.

- **Approximate sparsity.** Unlike IRLS, which tends to produce components that are exactly zero thanks to the weighted least-squares structure, the DSM yields solutions where “small” components are close to zero but rarely exactly zero. A post-processing thresholding step may be needed to enforce exact sparsity below a chosen tolerance.

Chapter 4

Comparison between the two Algorithms

Chapter 5

Implementation

Chapter 6

Results

Chapter 7

Conclusions

prova